GUIDE 20 · FREE · @AI_WITH_MUDIT

Solid, Scalable Software with *AI Skills*

Your MVP works. Now make it solid. Which skills to use, where to get them, and exactly when to run each one.

1 IDEA · 2 DESIGN · 3 PLAN · 4 BUILD · 5 BUG · 6 CHECK · 7 SHIP

@ai_with_mudit

The guide from the Day 113 reel · DM "scale"

Sources checked
15 October 2026
Claude Code · Codex

Contents

Read it top to bottom once. After that, the cheat sheet on the last page is all you need.

0	Start here	who this is for
1	Why your MVP breaks	the 5 missing pieces
2	Where to find skills	the shortlist + staying safe
3	How to install them	copy-paste commands
4	When to use which skill	the 7 stops (the real guide)
5	Three more skills	council, monitoring, load tests
6	Build it like a building	the rules block for CLAUDE.md / AGENTS.md
7	Advanced: a whole AI team	BMAD-METHOD · my VDP
8	Common mistakes	avoid these
9	Cheat sheet	screenshot this page

0 Start here

This guide is the promise from my Day 113 reel.

In the reel I said: building MVPs and demos with AI has become easy, but creating a solid, scalable architecture is still somewhat complex. To do it, you need to use multiple skills. And there were three questions: **where do you find them, how do you use them, and where do you use them.** This guide answers all three.

This is for you if...

You've already built something with Claude Code or Codex: an app, a tool, a demo. It works. But every new feature breaks two old ones, and the code is getting hard to follow.

Not there yet?

If you've never run a coding agent, read **guide 11 (Coding Agents)** first, then come back. If you want to write your own skills, that's **guide 04**.

HOW THIS WAS CHECKED

Every install command comes from the project's official README or docs, checked on **15 October 2026**, with links at the end. A separate AI agent, one that didn't write this guide, then audited the claims against the actual skill files. These libraries update almost every week, so glance at the README before you install.

1 Why your MVP breaks

An MVP is a ground floor with no foundation. It stands. Until you add the second floor.

When you tell an agent "build me this app", it does everything in one breath: thinking, design, code and testing, all in one chat. That's perfect for a demo. But software is like a building. Add the next floor without a foundation and the walls crack.

A vibe-coded MVP is almost always missing these five things:

What's missing	What happens
A spec. The agent builds what it *understood*	You get something close to what you meant, but not quite it
A plan. It jumps straight into code	Everything in one file, everything tangled with everything
Tests	A new feature silently breaks an old one
Review. The agent marks its own work "done"	Bugs and security holes show up in production
Standards. A new style every session	By week three nobody, including you, can read the code

THE IDEA IN ONE LINE

One skill for every missing step. A skill is a file that teaches the agent how to do one step properly, so you don't have to explain it every time.

2 Where to find skills

There are thousands of skills online. Most are noise, and some are dangerous. Here's my shortlist.

Superpowers

`obra/superpowers · MIT`

Skills for thinking, planning, test-driven development, debugging and review. They **trigger by themselves**.

`BASE` **Every project**

gstack

`garrytan/gstack · MIT`

A full engineering team as slash commands: plan reviews, QA, security audit, shipping.

`PLAN + SHIP` **Design and release**

interrogate (mine)

`Life696969/interrogate · MIT`

Interviews you before any code and writes `spec.md`. Later, a fresh session audits the build against it.

`SPEC + AUDIT` **Every new feature**

prompt-forge (mine)

`Life696969/prompt-forge · MIT`

Asks you 7 multiple-choice questions and turns a messy idea into one exact prompt.

`ONE-OFFS` **Scripts, single tasks**

Anthropic skills

`anthropics/skills · mostly Apache 2.0`

Official examples: `webapp-testing`, `frontend-design`, `mcp-builder`, `skill-creator`.

`OPTIONAL` **When you need them**

+ three more

`council · monitoring · load tests`

For big decisions and for production. They're in **section 5**.

`LATER` **Once the basics run**

HONEST NOTE

These five cover the **workflow**, every step from spec to ship. But no skill *builds* your production setup for you: monitoring, load tests, backups, CI and cost tracking. Some skills ask about these. Section 5 adds tools for two of them, and section 6 covers the rest as rules you follow yourself.

Looking for more skills?

In order of trust, highest first:

1. **Claude Code's `/plugin` command.** The official marketplace (`claude-plugins-official`) is already added; open the Discover tab. Website: claude.com/marketplace/plugins.
2. **anthropics/skills** (Claude) and **openai/plugins** (Codex). Official examples. The older `openai/skills` repo is deprecated.

3. **skills.sh** (by Vercel). Ranked by install count across about 20 agents. Install with `npx skills add <owner/repo>`. Popular doesn't mean safe: the security scan below covered this directory too.
4. **Awesome lists:** `hesreallyhim/awesome-claude-code`, `ComposioHQ/awesome-claude-skills`, `VoltAgent/awesome-agent-skills`. Good for discovery, but nobody has vetted every entry.

READ THIS BEFORE YOU INSTALL ANYTHING

A skill isn't just text. It can include scripts that run on your laptop with your permissions. Anthropic's own docs say a plugin can "execute arbitrary code on your machine". In February 2026, Snyk scanned 3,984 public skills (from ClawHub and skills.sh): **13.4% had at least one critical issue**, including malware, prompt injection and leaked secrets.

- Only install from sources you know (the list above).
- **Read** the `SKILL.md` and every script first. Ask your agent: *"Read every file in this skill folder and tell me what it runs, and whether it makes network calls or touches secrets."*
- For plugins, check `hooks/hooks.json`, `.mcp.json` and `bin/`.
- Don't install 50 skills at once. Fewer is better.

3 How to install them

Ten minutes, once. Then they're there for every project.

Two ways a skill runs

Automatically

The agent reads your request, matches it to a skill's `description` and loads it on its own. Superpowers works like this.

By name

You type `/skill-name` (Claude Code) or `$skill-name` (Codex). gstack is mostly used like this, though some of its skills (like `/ship` and `/qa`) suggest themselves at the right moment.

Agent	Where skills live
Claude Code	`~/.claude/skills/<name>/SKILL.md` (all projects) or `<project>/.claude/skills/` (one project)
Codex	`~/.agents/skills/<name>/SKILL.md` or `<project>/.agents/skills/`

On Windows, `~` means `C:\Users\your-name`.

1 Superpowers

`install this first · v7.0.0 (Oct 2026) · MIT`

Inside Claude Code:

```
/plugin install superpowers@claude-plugins-official
```

Codex CLI: type `/plugins`, search "superpowers", choose Install Plugin. **Cursor:** `/add-plugin superpowers` in Agent chat. **Gemini CLI:** `gemini extensions install https://github.com/obra/superpowers`.

Nothing else to do. The agent checks for a relevant skill before every task. To call one by name, say `superpowers:brainstorming`.

2 gstack

needs Git + Bun 1.4.2+ (and Node.js on Windows) · MIT

```
git clone --single-branch --depth 1 https://github.com/garrytan/gstack.git
~/.claude/skills/gstack
cd ~/.claude/skills/gstack && ./setup
```

On Windows, run it in Git Bash or WSL. For Codex: `./setup --host codex` (experimental; Claude Code has full support). Update with `/gstack-upgrade`. A command not showing up? Run `./setup --status`. Windows Smart App Control can block gstack's unsigned files; the README covers it.

3 interrogate + prompt-forge

mine · one folder each · no keys, no network · MIT

```
# get them
git clone https://github.com/Life696969/interrogate.git
git clone https://github.com/Life696969/prompt-forge.git

# Claude Code
mkdir -p ~/.claude/skills
cp -r interrogate/interrogate ~/.claude/skills/interrogate
cp -r prompt-forge/prompt-forge ~/.claude/skills/prompt-forge

# Codex (also read by Copilot CLI and Gemini CLI)
mkdir -p ~/.agents/skills
cp -r interrogate/interrogate ~/.agents/skills/interrogate
cp -r prompt-forge/prompt-forge ~/.agents/skills/prompt-forge
```

Windows PowerShell:

```
New-Item -ItemType Directory -Force "$env:USERPROFILE\.claude\skills" | Out-Null
Copy-Item -Recurse interrogate\interrogate "$env:USERPROFILE\.claude\skills\interrogate"
Copy-Item -Recurse prompt-forge\prompt-forge "$env:USERPROFILE\.claude\skills\prompt-
forge"
```

For Codex on Windows, use `.agents\skills` instead of `.claude\skills`. prompt-forge writes one file only: `~/prompt-forge/preferences.md`, where it remembers your choices. Delete it any time.

Check it works: open your agent, type `interrogate` plus one line about what you want built. The first reply should be a single question. That's it.

4 Anthropic skills

optional · mostly Apache 2.0

```
/plugin marketplace add anthropics/skills
/plugin install example-skills@anthropic-agent-skills
```

The useful ones here: `webapp-testing` (test your web app with Playwright), `frontend-design` (UI that doesn't look generic), `mcp-builder` (your own MCP server), `skill-creator` (build and test your own skills). Anthropic calls these demos, so test before you trust.

4 When to use which skill

Installing takes ten minutes. Knowing *which skill to run when* is the real skill.

Run a skill at the wrong time and you waste an hour. Run it at the right time and you get the step a senior engineer would have done. Every new feature goes through these 7 stops:

1	2	3	4	5	6	7
IDEA	DESIGN	PLAN	BUILD	BUG	CHECK	SHIP
interrogate	brainstorming plan reviews	writing-plans	subagents + TDD	systematic-debugging	fresh audit /review /cso	/ship

1 Idea → spec

`interrogate`

Why: the agent builds what it understood, not what you pictured. Close that gap before any code exists.

Say:

```
interrogate. I want a "share list" feature in my todo app, anyone with the link can view it
```

It asks one question at a time. Give it a vague answer and it pushes back once ("'fast' is a feeling, how many seconds?"). Then it plays your idea back in a short summary ("is this what you had in mind? yes / no"). On yes, it writes `spec.md` with Must have, Must never, Done means and Out of scope, **and stops.** No code until you've read the spec and said `build`.

> **THE TIP THAT MAKES IT SCALABLE**
>
> interrogate doesn't ask for scale numbers on its own. Give them with your idea: how many users, how much data, how fast. For example, "1,000 users at once, pages load in 2 seconds". They go into the spec, and the design is built around them.

Brand-new product, and the question is whether to build it at all? Run gstack's `/office-hours` first. It challenges the idea itself.

When to use prompt-forge instead: one-off jobs where you just need one perfect prompt, like "a script that renames my clips by recording date". Say `prompt forge`, answer 7 multiple-choice questions, get the exact prompt.

2 Design: where "scalable" gets planned

`brainstorming · /plan-eng-review · /plan-ceo-review`

Why: changing architecture after the code exists costs ten times more. This is exactly the step an MVP skips.

1. **Superpowers** `brainstorming` runs by itself when you say "let's design this feature". It asks what and why, plays its understanding back, and sizes the process to the job: a small task gets a small process. It clarifies **intent**; the next two steps handle architecture.
2. **gstack** `/plan-eng-review` reviews the architecture, component boundaries, data flow, edge cases, test coverage and performance (N+1 queries, caching) and writes the decisions into the plan.
3. **Users or data involved? Also run** `/plan-ceo-review` and pick "HOLD SCOPE". It has the best scalability checklist in this whole set: what breaks first at 10× and 100× load, whether migrations are safe, the rollback plan, and logs, metrics and alerts.

Keep in mind: a plan review gives you a **written plan**, not a lock. The agent can still drift during the build, which is why `/review` in stop 6 compares the code against the plan. Big feature? `/autoplan` runs the CEO, design, developer-experience and engineering reviews in one go. Torn between two architectures? That's what the council in section 5 is for.

3 Plan

`writing-plans`

Superpowers `writing-plans` breaks the design into small, testable tasks. The plan records **decisions** (function signatures, test assertions, exact values), not full code, so the agent doesn't wander during the build.

4 Build

`subagent-driven-development · TDD`

- **`subagent-driven-development`:** each task goes to a fresh subagent. After every task there's a review (does it match the spec, and is the code quality okay?), and a bigger review of the whole branch at the end.
- **`test-driven-development`:** write a failing test first, then the code, then clean up. Red, green, refactor. This is what stops "new feature, old one broke", because the old tests start failing. TDD focuses on unit tests; the tests that cross components (API to database, a full user flow) come from `/plan-eng-review`'s test plan and `/qa`. You need both.
- **`using-git-worktrees`:** a separate folder and branch for the feature, so your main code stays untouched.

Safety: gstack's `/careful` warns before destructive commands, and `/guard` adds a lock that keeps edits inside one folder. Keep them on when you're working near production.

5 A bug shows up

`systematic-debugging · /investigate`

The agent's habit: see an error, change something immediately, say "fixed!". Often it just hid the symptom.

Superpowers `systematic-debugging` kicks in when you report a bug. Four phases: find the root cause, analyse the pattern (compare with code that works), form a hypothesis and test it, then implement the fix. The iron law: **no fix without the root cause.** gstack's `/investigate` works the same way and stops to ask you after three failed attempts.

6 Check: the most important stop

`fresh audit · /review · /cso · /qa`

1. `verification-before-completion` (superpowers, automatic): the agent only says "done" after it has actually run the thing.
2. `interrogate audit` **in a FRESH session.** New window, new chat, one that never saw the build. It reads `spec.md`, runs the project, and writes `audit.md` with evidence for every line: the spec line, the `file:line`, the command it ran and what it saw. Ask for an audit in the builder's own chat and it refuses. That's the feature. (Guide 19, "The Second Agent Check", explains why.)
3. **gstack:**
   - `/review`: catches production bugs and over-built code. Its specialists also check database migrations (can it roll back? is data safe?) and API changes (anything breaking?), and compare the code against the plan.
   - `/cso`: a security audit (secrets, dependencies, OWASP Top 10). For now it reads your code; it doesn't attack the running app. If you have logins, payments or user data, never skip it. On Windows it needs Visual Studio 2022 Build Tools (Desktop C++); without them parts of the audit come back "not assessed".
   - `/qa` finds bugs and fixes them; `/qa-only` just reports them and touches no code.
   - `/health`: scores your type checker, linter, tests and dead code from 0 to 10. Run it weekly and watch the trend.

Any ✗ in `audit.md`? Give that file back to the builder, get it fixed, then audit again in **another** fresh window. Repeat until there are no ✗.

7 Ship

`finishing-a-development-branch · /ship`

- `finishing-a-development-branch` (superpowers): runs the tests first, then asks whether to merge, open a PR or keep the branch. To throw a branch away you have to say so explicitly; it never does that on its own.
- **gstack** `/ship`**:** checks tests and docs, pushes, opens the PR. `/land-and-deploy` merges and verifies production. `/canary` keeps watch after the deploy.

5 Three more skills

One for big decisions. Two for the production gaps no workflow skill covers.

A LLM Council: for the big decisions

`aiwithremy/claude-skills-llm-council · ~2.5k stars`

When: stop 2, when you're torn between two architectures. "Postgres or a document database?", "one service or split it?", "build auth ourselves or use a provider?"

What it does: it turns one question into five independent expert opinions, then a verdict. Five advisor subagents (including a Contrarian and an Outsider) answer in parallel. Five more review each other's answers **anonymously**, and a Chairman writes the final call. It's the LLM-council idea from Andrej Karpathy, built inside Claude with different thinking lenses instead of different models.

Install: it's one `SKILL.md` file, with no scripts and no API keys. Read it first (it's short), then save it as:

```
~/.claude/skills/llm-council/SKILL.md      # Claude Code
~/.agents/skills/llm-council/SKILL.md      # Codex (plain SKILL.md, should work; the
README only names Claude)
```

Use: say `council this:` plus your decision and the options. Give it your `spec.md` so the advisors argue about *your* project, not a generic one.

KNOW BEFORE YOU USE

- One council is about 11 subagent runs, so it uses noticeably more of your plan. Save it for decisions that are expensive to undo.
- The repo has **no license file**, so formally no rights are granted. Using it for your own work is low-risk; don't republish or sell it. It's a community project, not a vendor one.
- It reads your workspace for context. Don't run it in a folder full of secrets.
- Want a second opinion from a *different* model? gstack's `/codex` asks OpenAI's Codex to review your plan or code. It needs the Codex CLI installed and signed in (`npm install -g @openai/codex`, then `codex login`).

B Sentry: know when production breaks

official · getsentry · MIT

When: before your first real users, and again whenever you add a new service. This fills the biggest gap in the audit: *observability*.

```
# Claude Code (official marketplace)
/plugin install sentry@claude-plugins-official

# Codex
codex plugin marketplace add getsentry/plugin-codex
codex plugin add sentry@sentry-plugin-marketplace
```

What you get: skills that set up the Sentry SDK in your project (errors, tracing, logs and more), debug a live production issue, fix unreadable stack traces, set up releases and create alerts, plus Sentry's hosted MCP server.

Use: after install, ask "set up Sentry in this project" and let the setup skill do the wiring.

> **KNOW BEFORE YOU USE**
>
> - You need a Sentry account. The free Developer plan is one user with 5k errors and 5 GB of logs a month; Team is $26/month.
> - The connection uses OAuth (no key to paste). Uploading source maps needs an auth token.
> - Error reports from your app go to Sentry's servers. Don't send user passwords or payment data in them.

C Grafana k6: test the load before your users do

`official · grafana · k6 is AGPL-3.0`

When: before launch, and after any change to a hot path. "What happens when 1,000 users arrive at once?" should be answered by a test, not a guess.

Install: first install k6 itself (v2.0 or newer, free and local). Then, in your project folder:

```
k6 x agent init claude-code --dry-run    # see what it will write first
k6 x agent init claude-code
```

Codex users: there's a `codex-cli` target, but it writes to `.codex/skills` and `.codex/mcp.json`, which current Codex docs don't list (Codex reads `.agents/skills` and `config.toml`). Treat it as unverified for now.

What you get: 11 skills, including `k6-test-planner`, `k6-smoke-test`, `k6-load-test` and `k6-perf-test-website`, plus a local k6 MCP server. Ask "plan a load test for the checkout API, 1,000 users" and run a **smoke test** before the big one.

FIX THE SETTINGS FILE AFTER INSTALL

- For Claude Code it writes `.claude/settings.local.json` with `"enableAllProjectMcpServers": true`. That auto-approves **every** MCP server in the project, not just k6. Delete that line.
- The same file also has an allow-list key spelled `enabledMcpJsonServers`. Claude Code's documented spelling is `enabledMcpjsonServers` (lowercase **j**), so fix it to `"enabledMcpjsonServers": ["k6"]`. Check the file again whenever you re-run `init`.
- The first run downloads the extension. Grafana Cloud (and its `gcx` CLI) is only needed for the three cloud skills: `k6-manage`, `k6-cloud-investigate-test` and `k6-trend-analysis`.
- Only load-test servers you own. Pointing a load test at someone else's site is an attack.

WHERE THEY FIT

Council → stop 2, for the hard decisions. **Sentry** → set up once, then rule 11 in section 6. **k6** → stop 6, before a launch or after a performance-sensitive change.

6 Build it like a building

"Build the first two floors, but lay the foundation for five."

That's the principle I run on every project. Build only what you need today, but structure it so the next feature joins like a new floor, without tearing the building down. No single skill teaches this, so here's a rules block. Paste it into your project's `CLAUDE.md` (Claude Code) or `AGENTS.md` (Codex). The agent reads it every session.

```
## Building rules (follow on every task)

1. Keep layers apart: core logic / flows / adapters (DB, APIs, files) / UI.
   Core never imports the UI or the database directly.
2. New item = new file. If adding one more screen, record type or
   integration means editing the core, the design is wrong. Build the
   extension point first (a registry, config or interface).
3. No magic values. Limits, URLs, keys and flags live in config.
4. Small modules. One file, one job.
5. Version your data. Schema changes ship with a migration; never break data.
6. Formatter + linter + tests from the first task, running in CI
   (e.g. GitHub Actions) on every PR. Not "later".
7. Catch errors at the boundaries (API, files, user input), not deep inside.
8. Tests for every layer: unit tests for core, integration tests for flows.
9. In an existing codebase, its style wins. No refactors nobody asked for.
10. Don't build speculative features. Strong foundation, but only the
    floors you need today.
11. Production: structured logs on every new code path and production
    errors in an error tracker. Automatic database backups with a restore
    you've tested once. A note of the monthly cost of every new service.
    Big decisions recorded as short files in docs/decisions/. Dependency
    update bots (Dependabot/Renovate) on.

At the end of every task, write one line: "Could the next item (screen /
record / integration) be added as new files, without changing the core?"
Yes or no, and why.
```

THE FLOOR CHECK

That last line is the most useful one. I call it the **floor check**. It forces the agent to think, on every task, about how the next floor will attach. The day the answer is "no" is the day to fix the architecture, while it's still cheap.

RULE 11 IS ON YOU

No skill *builds* rule 11 for you. Skills ask about it (`/plan-ceo-review` asks about logs, rollback and backups), and section 5 gives you tools for monitoring and load tests, but the setup is yours.

7 Advanced: a whole AI team

One developer with skills goes a long way. Big projects want roles.

On bigger projects I run three separate roles: one agent that talks to me and writes the spec, one that only builds and never talks to me, and a fresh tester that only tries to break things and writes a PASS/FAIL report. They talk through files, never through chat.

COMING SOON: MY VDP

I've built my own version: **VDP, the Vision-Developer Protocol.** Coding standards are always on and every task gets a floor check. **I'll be releasing it soon.** Follow along and I'll announce it on a reel.

Until then, the closest public alternative is **BMAD-METHOD** (`bmad-code-org/BMAD-METHOD`, MIT, about 54k stars):

- Separate roles for planning and building (the v6 docs list personas like Analyst, PM, Architect, Developer and UX).
- Work lives in files: stories, plans, review notes.
- The docs recommend running each workflow in a fresh chat.
- A separate QA step, plus a dedicated Test Architect module for bigger projects.

Install the stable v6.12 (needs Node 20.12+ and Python 3.10+):

```
npx bmad-method install
```

V7 preview: `/plugin marketplace add bmad-code-org/bmad-plugins` in Claude Code, or `codex plugin marketplace add bmad-code-org/bmad-plugins` in Codex. As of 15 October 2026, V7 is in alpha with a stable release days away, so check the README and pick a version before you install. Names and folders may change.

TWO VDP RULES TO ADD YOURSELF

(1) The building agent doesn't make product decisions with you; it works only from the spec. (2) The tester always runs in a new window and writes its PASS/FAIL report to a file. Guide 19 has copy-paste prompts for both.

Only want a strong spec, not roles? Try **GitHub Spec Kit** (`github/spec-kit`): `/speckit-constitution` (your project's rules), `/speckit-specify`, `/speckit-plan`, `/speckit-tasks` and `/speckit-implement`. Its spec template has a required "User Scenarios & Testing" section with Given/When/Then acceptance scenarios. Install with `uv tool install specify-cli`, then `specify init <name> --integration claude` (or `codex`) in your project.

8 Common mistakes

Every one of these costs days. Each takes two minutes to avoid.

> **Skipping design.** "It's a small feature", so you go straight to building. Three days later you need the same logic somewhere else, and it's all in one file. Let brainstorming run even on small features; it picks a small process by itself.

> **Letting the builder audit itself.** "Check your work" in the same chat. The agent rereads its own work and says everything's fine. Audits always happen in a fresh session.

> **Installing every skill at once.** When two skills have similar descriptions, the agent runs the wrong one. Start with just superpowers + interrogate. Use them for a week. Then add gstack.

> **Saying `build` without reading the spec.** The thing you wanted might be sitting under "Out of scope". Reading the spec takes two minutes and saves two days.

> **Leaving tests for "later".** Later never comes. Keep the TDD skill on from day one.

> **Installing a skill without reading it.** A skill can carry a script that runs every session. Reread the security box in section 2.

9 Cheat sheet

Screenshot this page.

Install once

- `/plugin install superpowers@claude-plugins-official`
- gstack: `git clone … ~/.claude/skills/gstack && ./setup`
- interrogate + prompt-forge: copy each folder into `~/.claude/skills/`
- Later: council `SKILL.md` · `/plugin install sentry@claude-plugins-official` · `k6 x agent init claude-code --dry-run`

Every feature

1. `interrogate <idea + scale numbers>` → read spec.md → `build`
2. "let's design this" → brainstorming · `/plan-eng-review` · `/plan-ceo-review` (HOLD)
3. writing-plans → small tasks
4. Build: subagents + TDD, `/careful` on
5. Bug? systematic-debugging: root cause first
6. NEW window: `interrogate audit`, then `/review` `/cso` `/qa-only` `/health`
7. `/ship`

Rules

- No build without reading the spec.
- Audits always in a fresh session.
- Floor check on every task.
- Read every file of a skill before installing it.
- Few skills first. One week. Then more.
- Logs, backups, CI, load tests: skills ask, you set them up.
- Big decision? `council this`. Launch? k6 smoke test first.

"You are where I was six months ago."

That's what I said in the reel. Back then I also thought building complex software with AI was easy. The difference wasn't the AI model. It was the **steps**: spec, design, plan, test, review, the same steps good software teams have followed for years. Skills just make sure the agent doesn't skip them.

Do one thing today: install superpowers and interrogate, and run your MVP's **next** feature through stops 1 to 7. One feature. You'll see the difference yourself.

@ai_with_mudit · Day 113 of sharing about AI. Free: read it, share it, credit me. Don't sell it.
Sources (15 October 2026): github.com/obra/superpowers · github.com/garrytan/gstack · github.com/anthropics/skills · github.com/openai/plugins · github.com/bmad-code-org/BMAD-METHOD · github.com/github/spec-kit · code.claude.com/docs/en/skills · code.claude.com/docs/en/plugins/security · snyk.io/blog/toxicskills-malicious-ai-agent-skills-clawhub · github.com/Life696969/interrogate · github.com/Life696969/prompt-forge · github.com/aiwithremy/claude-skills-llm-council · github.com/getsentry/plugin-claude · github.com/getsentry/plugin-codex · sentry.io/pricing · grafana.com/docs/k6/latest/set-up/configure-ai-assistant · github.com/grafana/xk6-subcommand-agent